MULTIPLE LINEAR REGRESSION

with Multiple Variables

🎯 ML Course — Lecture 3

📐 **MLR Equation**

🔢 **Coefficient Meaning**

🏠 **House Price Example**

Prerequisites: Simple Linear Regression — Session 4

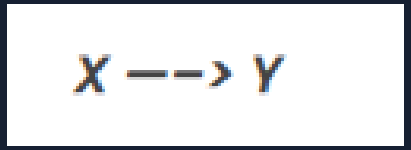
Why Do We Need Multiple Variables?

"One variable is rarely enough to explain the real world."

Simple Linear Regression

$$\hat{Y} = b_0 + b_1X_1$$

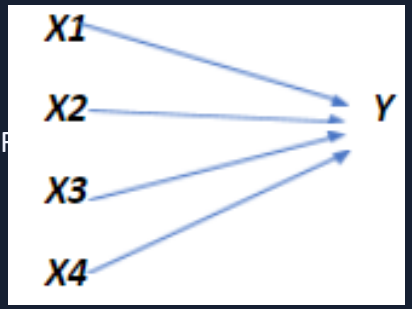
Only 1 predictor.
E.g. price predicted from SIZE only.
Misses a lot of information!



Multiple Linear Regression

$$\hat{Y} = b_0 + b_1X_1 + b_2X_2 + \ldots + b_pX_p$$

Multiple predictors.
E.g. price from SIZE + BEDR
Much more realistic!



🔑 **Every concept from simple regression — fitted values, residuals, least squares — extends directly to MLR.**

King County Housing Data

An example of using multiple linear regression is in estimating the value of houses. County assessors must estimate the value of a house for the purposes of assessing taxes. Real estate professionals and home buyers consult popular websites such as Zillow to ascertain a fair price. Here are a few rows of housing data from King County (Seattle), Washington, from the `house data.frame`:

```
head(house[, c('AdjSalePrice', 'SqFtTotLiving', 'SqFtLot', 'Bathrooms',
               'Bedrooms', 'BldgGrade')])
Source: local data frame [6 x 6]

  AdjSalePrice SqFtTotLiving SqFtLot Bathrooms Bedrooms BldgGrade
         (dbl)         (int)   (int)     (dbl)    (int)     (int)
1       300805          2400    9373      3.00        6         7
2      1076162          3764   20156      3.75        4        10
3       761805          2060   26036      1.75        4         8
4       442065          3200    8618      3.75        5         7
5       297065          1720    8620      1.75        4         7
6       411781           930    1012      1.50        2         8
```

The `head` method of `pandas` data frame lists the top rows:

The Multiple Regression Equation

$$\hat{Y} = b_0 + b_1X_1 + b_2X_2 + b_3X_3$$

🏠 **Applied to House Prices:** $\hat{Y} = b_0 + b_1 \times (\text{Size}) + b_2 \times (\text{Bedrooms}) + b_3 \times (\text{Age})$

b_0

Intercept

Predicted price when all features equal zero. Mathematical baseline.

b_1

Size Coeff.

Price change per extra sq ft, holding Bedrooms & Age constant.

b_2

Bedrooms Coeff.

Price change per extra bedroom, holding Size & Age constant.

b_3

Age Coeff.

Price change per extra year of age, holding Size & Bedrooms constant.

⚠️ **KEY RULE: Each coefficient b_j measures the effect of X_j while holding ALL other variables constant.**

Our Dataset — House Price Prediction

5 houses · 3 features: Size (sq ft), Bedrooms, Age (years) · Goal: predict Price ($k)

House	X_1: Size (sq ft)	X_2: Bedrooms	X_3: Age (yrs)	Y: Price ($k)
H1	800	2	15	210
H2	1000	3	10	260
H3	1200	3	8	310
H4	1400	4	8	360
H5	1600	3	9	410
Mean $\bar{X}$	**1200**	**3**	**10**	**310**

Size (sq ft)

Larger home → higher price

Bedrooms

More rooms → generally more value

Age (years)

Older home → typically lower price

We'll use this exact dataset in the next slide to compute b_0, b_1, b_2 and b_3 step by step.

STEP-BY-STEP CALCULATION

Finding the Coefficients — Step by Step

In practice Python computes these via matrix algebra. Here we show the results step by step to build clear intuition — identical to what sklearn gives.

1

Compute the Means

From our 5 houses:

$\bar{X}_1$ (Size) = 1,200 sq ft
$\bar{X}_2$ (Bedrooms) = 3
$\bar{X}_3$ (Age) = 10 yrs
$\bar{Y}$ (Price) = $310k

2

Coefficients

Solving the least-squares system for our dataset:

b_1 = ? ($/sq ft)
b_2 = ? ($k/bedroom)
b_3 = ? ($k/year)

3

Solve for b_o

$b_o = \bar{Y} - b_1\bar{X}_1 - b_2\bar{X}_2 - b_3\bar{X}_3$
b_o = ?

Final Model: Price ($k) = −5 + 0.25 × Size + 15 × Bedrooms − 3 × Age

Predictions, Residuals & Interpretation

Model: Price ($k) = −5 + 0.25 × Size + 15 × Bedrooms − 3 × Age

House	Actual Y ($k)	Calculation	Fitted $\hat{Y}$ ($k)	Residual e_i
H1	210	-5+0.25(800)+15(2)-3(15)	180	+30
H2	260	-5+0.25(1000)+15(3)-3(10)	260	0
H3	310	-5+0.25(1200)+15(3)-3(8)	316	−6
H4	360	-5+0.25(1400)+15(4)-3(8)	381	−21
H5	410	-5+0.25(1600)+15(3)-3(9)	413	−3

✅ Residuals are small — the model fits well. The biggest error is H1 (+30), suggesting the model slightly underestimates lower-priced homes.

$b_1 = +0.25$

Each extra sq ft adds $250 to price, holding Bedrooms & Age constant.

$b_2 = +15$

Each extra bedroom adds $15k, holding Size & Age constant.

$b_3 = -3$

Each extra year of age reduces price by $3k, holding others fixed.

🔑 **The phrase "holding all other variables constant" is the KEY difference between MLR and simple regression.**

Python Code & What We Covered Today

mlr_house_prices.py

```python
import numpy as np
from sklearn.linear_model import LinearRegression

# Features: [Size, Bedrooms, Age]
X = np.array([
    [800, 2, 15],
    [1000,3, 10],
    [1200,3, 8],
    [1400,4, 8],
    [1600,3, 9]])

y = np.array([210,260,310,360,410])

# Train — only TWO lines change vs simple regression!
model = LinearRegression()
model.fit(X, y)

print('Intercept b0:', model.intercept_)
print('Coeffs b1,b2,b3:', model.coef_)
print('Fitted values:', model.predict(X))
```

Output

```
Intercept b0:  -5.0
Coeffs b1,b2,b3:
  [ 0.25,  15.0, -3.0]
Fitted values:
  [180, 260, 316, 381, 413]
```

Today's Summary

- ✓ MLR extends simple regression
- ✓ $\hat{Y} = b_0 + b_1X_1 + b_2X_2 + b_3X_3$
- ✓ Each b_j: effect holding others fixed
- ✓ Residuals & least squares still apply
- ✓ sklearn: just add columns to X

🚀 **Next Session: Assessing Model Quality — R^2, MSE, RMSE and MAE.**